

# Assignment: AI-Powered Task & Knowledge Management System

Duration: 1.5 Days

## Objective

Build a **minimal working system (MVP)** where:

- Admin uploads documents and assigns tasks
- Users search documents using AI and complete tasks

We are evaluating your ability to **design systems, not just write code.**

## Problem Statement

Create a system where:

- Admin builds a **knowledge base (documents)**
- Users use **AI-powered search** to find answers
- Users complete assigned tasks using that knowledge

## Mandatory Requirements (MUST HAVE)

### 1. Authentication & RBAC

- Implement **JWT authentication**
- Roles:
  - Admin
  - User
- Protect APIs based on role

### 2. Database (MySQL Mandatory)

Design proper relational schema with **PK/FK**

Minimum tables:

- users
- roles
- tasks
- documents

- activity\_logs

### 3. Document Upload

- Upload `.txt`, `pdf` files
- Store document metadata
- Prepare data for AI processing

### 4. AI Feature (Core Requirement)

Implement **semantic search**:

- Convert text → embeddings
- Store in vector DB (FAISS/Chroma)
- Query → embedding → retrieve results

Do NOT rely only on LLM APIs  
Core logic must be implemented

### 5. Task Management

- Admin:
  - Create and assign tasks
- User:
  - View tasks
  - Update status:
    - Pending → Completed

### 6. Dynamic Filtering API

At least one API must support filtering:

Example:

- `/tasks?status=completed`
- `/tasks?assigned_to=1`

### 7. Activity Logging (Mandatory)

Track key actions:

- Login
- Task update
- Document upload
- Search

## 8. Basic Analytics

Show at least:

- Total tasks
- Completed vs pending
- Most searched queries (or count)

## Required APIs

- `/auth/login`
- `/tasks`
- `/documents`
- `/search`
- `/analytics`

## Constraints (VERY IMPORTANT)

- Use **Python (FastAPI)** for backend
- Use **React.js** for frontend
- Use **MySQL** (with relationships)
- Implement JWT authentication
- Implement RBAC
- Implement embedding-based search
- Write clean, structured, scalable code

## Evaluation Criteria

We will evaluate your work based on:

### Backend (Python / FastAPI)

- API design
- Clean architecture
- Proper use of models, services, and controllers
- Database design (relations, normalization)

### Frontend (React.js)

- UI structure and usability
- API integration
- State management
- Clean component design

## **AI Implementation**

- Correct use of embeddings
- Semantic search quality
- Vector DB usage (FAISS/Chroma)
- Not over-relying on external APIs

## **Overall Code Quality**

- Readability
- Folder structure
- Naming conventions
- Maintainability

## **Submission**

- GitHub repository
- README:
  - Setup steps
  - Tech stack
  - Short explanation video (optional)
  - Output screenshots (must)